Guía de Ejercicios Resueltos

Semana 5 — Coordinación entre clases en un entorno concurrente

Desarrollo Orientado a Objetos II — PRY2203

10 ejercicios prácticos comentados en Java

Objetivo de esta guía: reforzar los mecanismos de sincronización trabajados en la semana 5: `synchronized`, `wait()/notify()`, `PriorityBlockingQueue`, `ReentrantLock`, `Semaphore`, `volatile` y `AtomicInteger`. Cada ejercicio incluye el enunciado, el código resuelto con comentarios en los métodos y partes clave, un ejemplo de salida por consola y una breve explicación del concepto aplicado.

#	Ejercicio	Eje principal
1	Contador compartido con método synchronized	synchronized (método)
2	Impresión ordenada con bloque synchronized	synchronized (bloque)
3	Buzón productor-consumidor	wait() / notify()
4	Cola de tareas por prioridad	PriorityBlockingQueue
5	Caja registradora con acceso exclusivo	ReentrantLock
6	Estacionamiento con cupos limitados	Semaphore
7	Bandera de apagado entre hilos	volatile
8	Generador de identificadores únicos	AtomicInteger
9	Impresora compartida con cola de prioridad y lock	PriorityBlockingQueue + ReentrantLock
10	Monitor de estado en tiempo real	volatile + hilo observador

Ejercicios resueltos

Ejercicio 1: Contador compartido con método synchronized

synchronized

Enunciado: Varios hilos incrementan un contador compartido 1000 veces cada uno. Sin protección, el resultado final es incorrecto por condiciones de carrera. Implementa una clase `ContadorSeguro` que use `synchronized` para asegurar el resultado correcto.

```
public class ContadorSeguro {

    // Variable compartida entre todos los hilos
    private int valor = 0;

    /**
     * Incrementa el contador de forma segura.
     * synchronized asegura que solo un hilo a la vez
     * pueda ejecutar este método sobre la misma instancia.
     */
    public synchronized void incrementar() {
        valor++;
    }

    public synchronized int getValor() {
        return valor;
    }

    public static void main(String[] args) throws InterruptedException {
        ContadorSeguro contador = new ContadorSeguro();

        // Tarea que incrementa el contador 1000 veces
        Runnable tarea = () -> {
            for (int i = 0; i < 1000; i++) {
                contador.incrementar();
            }
        };

        Thread h1 = new Thread(tarea);
        Thread h2 = new Thread(tarea);

        h1.start();
        h2.start();
        h1.join();
        h2.join();

        System.out.println("Valor final del contador: " + contador.getValor());
    }
}
```

Salida por consola:

```
Valor final del contador: 2000
```

¿Qué se aplicó? Al marcar `incrementar()` como `synchronized`, cada hilo debe esperar a que el otro termine de modificar `valor` antes de entrar al método, evitando que ambos incrementos se pierdan entre sí (condición de carrera).

Ejercicio 2: Impresión ordenada con bloque synchronized

synchronized

Enunciado: Dos hilos imprimen mensajes en consola. Se necesita que cada mensaje se imprima completo (sin mezclarse con el de otro hilo), usando un bloque `synchronized` sobre un objeto candado en lugar de sincronizar todo el método.

```
public class ImpresorSeguro {

    // Objeto usado exclusivamente como candado (lock)
    private final Object candado = new Object();

    public void imprimirMensaje(String hilo, String mensaje) {
        // Solo el bloque crítico queda protegido, no todo el método
        synchronized (candado) {
            System.out.print("[" + hilo + "] ");
            System.out.print(mensaje);
            System.out.println(" (fin)");
        }
    }

    public static void main(String[] args) {
        ImpresorSeguro impresor = new ImpresorSeguro();

        Runnable tareaA = () -> impresor.imprimirMensaje("Hilo-A", "Reporte de ventas");
        Runnable tareaB = () -> impresor.imprimirMensaje("Hilo-B", "Reporte de stock");

        new Thread(tareaA).start();
        new Thread(tareaB).start();
    }
}
```

Salida por consola (orden puede variar entre ejecuciones):

```
[Hilo-A] Reporte de ventas (fin)
[Hilo-B] Reporte de stock (fin)
```

¿Qué se aplicó? El bloque `synchronized(candado)` protege solo la sección crítica (las tres impresiones), evitando que un hilo interrumpa a otro a mitad de un mensaje. Usar un objeto candado dedicado es más flexible que sincronizar el método completo.

Ejercicio 3: Buzón productor-consumidor

wait() / notify()

Enunciado: Implementa un Buzon donde un hilo productor deposita un mensaje y un hilo consumidor espera hasta que haya un mensaje disponible, usando `wait()` y `notify()`.

```
public class Buzon {

    private String mensaje;
    private boolean disponible = false;

    /**
     * El consumidor llama a este método.
     * Si no hay mensaje, el hilo se bloquea con wait()
     * hasta que otro hilo llame a notify().
     */
    public synchronized String recibir() throws InterruptedException {
        while (!disponible) {
            wait(); // libera el monitor y espera
        }
        disponible = false;
        notify(); // avisa que el buzón quedó libre
        return mensaje;
    }

    /**
     * El productor llama a este método para dejar un mensaje.
     */
    public synchronized void enviar(String msg) throws InterruptedException {
        while (disponible) {
            wait(); // espera si el buzón sigue ocupado
        }
        mensaje = msg;
        disponible = true;
        notify(); // despierta al consumidor
    }

    public static void main(String[] args) {
        Buzon buzon = new Buzon();

        Thread productor = new Thread(() -> {
            try {
                buzon.enviar("Pedido #1 listo");
                System.out.println("[Productor] Mensaje enviado");
            } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        });

        Thread consumidor = new Thread(() -> {
            try {
                String recibido = buzon.recibir();
                System.out.println("[Consumidor] Recibido: " + recibido);
            } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        });

        consumidor.start();
        productor.start();
    }
}
```

Salida por consola:

```
[Productor] Mensaje enviado
[Consumidor] Recibido: Pedido #1 listo
```

¿Qué se aplicó? El consumidor queda en espera con `wait()` mientras no haya mensaje, sin consumir CPU en un bucle activo. El uso de `while` (en lugar de `if`) protege contra los "despertares espurios". `notify()` reactiva al hilo que espera sobre el mismo monitor.

Ejercicio 4: Cola de tareas por prioridad

PriorityBlockingQueue

Enunciado: Crea una clase `Tarea` comparable por prioridad y demuestra que una `PriorityBlockingQueue` entrega siempre primero la tarea más urgente, sin importar el orden de llegada.

```
public class Tarea implements Comparable<Tarea> {
    String nombre;
    int prioridad; // 1 = más urgente

    public Tarea(String nombre, int prioridad) {
        this.nombre = nombre;
        this.prioridad = prioridad;
    }

    // Define el orden dentro de la cola: menor número = mayor prioridad
    @Override
    public int compareTo(Tarea otra) {
        return Integer.compare(this.prioridad, otra.prioridad);
    }

    @Override
    public String toString() {
        return nombre + " (prioridad " + prioridad + ")";
    }

    public static void main(String[] args) {
        PriorityBlockingQueue<Tarea> cola = new PriorityBlockingQueue<>();

        // Se agregan en orden de llegada, no de urgencia
        cola.add(new Tarea("Backup nocturno", 3));
        cola.add(new Tarea("Caída de servidor", 1));
        cola.add(new Tarea("Actualizar reporte", 2));

        // poll() siempre retorna la de mayor prioridad (menor número)
        while (!cola.isEmpty()) {
            System.out.println("Atendiendo: " + cola.poll());
        }
    }
}
```

Salida por consola:

```
Atendiendo: Caída de servidor (prioridad 1)
Atendiendo: Actualizar reporte (prioridad 2)
Atendiendo: Backup nocturno (prioridad 3)
```

¿Qué se aplicó? `PriorityBlockingQueue` es thread-safe y ordena automáticamente los elementos usando `compareTo()`. Aunque las tareas se insertaron en otro orden, siempre se procesa primero la más urgente.

Ejercicio 5: Caja registradora con acceso exclusivo

ReentrantLock

Enunciado: Varios cajeros (hilos) retiran dinero de una caja compartida. Usa `ReentrantLock` para asegurar que solo un cajero acceda al saldo a la vez, evitando retiros inconsistentes.

```
public class CajaRegistradora {

    private int saldo;
    private final ReentrantLock lock = new ReentrantLock();

    public CajaRegistradora(int saldoInicial) {
        this.saldo = saldoInicial;
    }

    /**
     * Retira un monto de forma exclusiva.
     * lock()/unlock() garantizan que ningún otro hilo
     * lea o modifique 'saldo' mientras se ejecuta el retiro.
     */
    public boolean retirar(String cajero, int monto) {
        lock.lock();
        try {
            if (monto > saldo) {
                System.out.println("[" + cajero + "] Fondos insuficientes");
                return false;
            }
            saldo -= monto;
            System.out.println("[" + cajero + "] Retiró " + monto + ". Saldo: " + saldo);
            return true;
        } finally {
            lock.unlock(); // siempre se libera, incluso si hay una excepción
        }
    }

    public static void main(String[] args) throws InterruptedException {
        CajaRegistradora caja = new CajaRegistradora(1000);

        Runnable retiroA = () -> caja.retirar("Cajero-A", 700);
        Runnable retiroB = () -> caja.retirar("Cajero-B", 700);

        Thread t1 = new Thread(retiroA);
        Thread t2 = new Thread(retiroB);
        t1.start();
        t2.start();
        t1.join();
        t2.join();
    }
}
```

Salida por consola:

```
[Cajero-A] Retiró 700. Saldo: 300
[Cajero-B] Fondos insuficientes
```

¿Qué se aplicó? `ReentrantLock` asegura acceso exclusivo a la sección crítica, igual que `synchronized`, pero con control explícito (`lock()`/`unlock()`) y la posibilidad de usar `tryLock()` o `Condition` para escenarios más avanzados.

Ejercicio 6: Estacionamiento con cupos limitados

Semaphore

Enunciado: Un estacionamiento tiene solo 2 cupos disponibles. Varios autos (hilos) intentan entrar al mismo tiempo. Usa `Semaphore` para limitar el acceso concurrente a la cantidad de cupos.

```
public class Estacionamiento {

    // Semáforo con 2 permisos: solo 2 hilos pueden estar "dentro" a la vez
    private final Semaphore cupos = new Semaphore(2);

    public void estacionar(String auto) {
        try {
            cupos.acquire(); // pide un permiso; espera si no hay disponibles
            System.out.println("[" + auto + "] Entró al estacionamiento");
            Thread.sleep(1000); // simula tiempo estacionado
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        } finally {
            System.out.println("[" + auto + "] Salió del estacionamiento");
            cupos.release(); // libera el permiso para otro auto
        }
    }

    public static void main(String[] args) {
        Estacionamiento parking = new Estacionamiento();
        for (int i = 1; i <= 3; i++) {
            String auto = "Auto-" + i;
            new Thread(() -> parking.estacionar(auto)).start();
        }
    }
}
```

Salida por consola (ejemplo):

```
[Auto-1] Entró al estacionamiento
[Auto-2] Entró al estacionamiento
[Auto-1] Salió del estacionamiento
[Auto-3] Entró al estacionamiento
[Auto-2] Salió del estacionamiento
[Auto-3] Salió del estacionamiento
```

¿Qué se aplicó? `Semaphore` controla cuántos hilos pueden acceder simultáneamente a un recurso limitado. `Auto-3` debe esperar hasta que `acquire()` obtenga un permiso liberado por `release()`.

Ejercicio 7: Bandera de apagado entre hilos

volatile

Enunciado: Un hilo trabajador se ejecuta en bucle hasta que el hilo principal indica que debe detenerse. Usa `volatile` para que el cambio sea visible de inmediato entre hilos.

```java
public class Trabajador implements Runnable {

    // volatile: garantiza que el cambio hecho por otro hilo sea visible aquí
    private volatile boolean activo = true;

    public void detener() {
        activo = false; // visible de inmediato para el hilo run()
    }

    @Override
    public void run() {
        int ciclos = 0;
        while (activo) {
            ciclos++;
            System.out.println("Trabajando... ciclo " + ciclos);
            try { Thread.sleep(500); } catch (InterruptedException e) { break; }
        }
        System.out.println("Trabajador detenido tras " + ciclos + " ciclos");
    }

    public static void main(String[] args) throws InterruptedException {
        Trabajador trabajador = new Trabajador();
        Thread hilo = new Thread(trabajador);
        hilo.start();

        Thread.sleep(1700); // deja correr el trabajo unos segundos
        trabajador.detener();
    }
}
```

Salida por consola:

```
Trabajando... ciclo 1
Trabajando... ciclo 2
Trabajando... ciclo 3
Trabajador detenido tras 3 ciclos
```

¿Qué se aplicó? Sin `volatile`, el hilo trabajador podría no notar nunca el cambio de `activo` a `false` (por optimizaciones de caché de cada núcleo), quedando en bucle infinito. `volatile` asegura visibilidad inmediata entre hilos.

Ejercicio 8: Generador de identificadores únicos

AtomicInteger

Enunciado: Varios hilos generan pedidos de forma concurrente y cada uno necesita un identificador único, sin usar `synchronized`. Usa `AtomicInteger`.

```
public class GeneradorId {

    // AtomicInteger permite incrementar de forma segura sin bloqueos explícitos
    private static final AtomicInteger contador = new AtomicInteger(1);

    /**
     * Obtiene el siguiente id disponible de forma atómica.
     * getAndIncrement() lee y suma en una sola operación indivisible.
     */
    public static int siguienteId() {
        return contador.getAndIncrement();
    }

    public static void main(String[] args) throws InterruptedException {
        Runnable generar = () -> {
            int id = siguienteId();
            System.out.println(Thread.currentThread().getName() + " generó pedido #" + id);
        };

        Thread t1 = new Thread(generar, "Camarero-1");
        Thread t2 = new Thread(generar, "Camarero-2");
        Thread t3 = new Thread(generar, "Camarero-3");

        t1.start(); t2.start(); t3.start();
        t1.join(); t2.join(); t3.join();

        System.out.println("Total de pedidos generados: " + (contador.get() - 1));
    }
}
```

Salida por consola (el orden puede variar):

```
Camarero-1 generó pedido #1
Camarero-3 generó pedido #2
Camarero-2 generó pedido #3
Total de pedidos generados: 3
```

¿Qué se aplicó? `AtomicInteger` garantiza que cada llamada a `getAndIncrement()` sea una operación atómica: ningún id se repite ni se salta, sin necesidad de `synchronized` ni `Lock`.

Ejercicio 9: Impresora compartida con cola de prioridad y lock

PriorityBlockingQueue + ReentrantLock

Enunciado: Combina `PriorityBlockingQueue` y `ReentrantLock` para simular una impresora compartida: los trabajos de impresión se ordenan por prioridad y el acceso a la cola se protege explícitamente.

```
public class ImpresoraCompartida {

    private final PriorityBlockingQueue<Tarea> cola = new PriorityBlockingQueue<>();
    private final ReentrantLock lock = new ReentrantLock();

    /**
     * Agrega un trabajo de impresión de forma protegida.
     */
    public void agregarTrabajo(Tarea trabajo) {
        lock.lock();
        try {
            cola.add(trabajo);
            System.out.println("Trabajo en cola: " + trabajo);
        } finally {
            lock.unlock();
        }
    }

    /**
     * Imprime el trabajo de mayor prioridad disponible.
     */
    public void imprimirSiguiente() {
        lock.lock();
        try {
            Tarea trabajo = cola.poll();
            if (trabajo != null) {
                System.out.println("Imprimiendo: " + trabajo);
            } else {
                System.out.println("No hay trabajos pendientes");
            }
        } finally {
            lock.unlock();
        }
    }

    public static void main(String[] args) {
        ImpresoraCompartida impresora = new ImpresoraCompartida();
        impresora.agregarTrabajo(new Tarea("Informe final", 2));
        impresora.agregarTrabajo(new Tarea("Contrato urgente", 1));
        impresora.agregarTrabajo(new Tarea("Volante", 3));

        impresora.imprimirSiguiente();
        impresora.imprimirSiguiente();
        impresora.imprimirSiguiente();
    }
}
```

Salida por consola:

```
Trabajo en cola: Informe final (prioridad 2)
Trabajo en cola: Contrato urgente (prioridad 1)
Trabajo en cola: Volante (prioridad 3)
Imprimiendo: Contrato urgente (prioridad 1)
Imprimiendo: Informe final (prioridad 2)
Imprimiendo: Volante (prioridad 3)
```

¿Qué se aplicó? El mismo patrón usado en `CocinaSincronizada` de la guía: la cola de prioridad ordena los trabajos automáticamente, mientras el `ReentrantLock` protege las operaciones de agregar y retirar frente a un entorno con múltiples hilos concurrentes.

Ejercicio 10: Monitor de estado en tiempo real

volatile + hilo observador

Enunciado: Crea un hilo `MonitorPedidos` que reporte cada cierto tiempo cuántos pedidos hay pendientes en la impresora del ejercicio anterior, y que pueda detenerse de forma segura desde otro hilo.

```
public class MonitorPedidos implements Runnable {

    private final PriorityBlockingQueue<Tarea> cola;
    // volatile: el cambio hecho por detener() debe ser visible en run()
    private volatile boolean activo = true;

    public MonitorPedidos(PriorityBlockingQueue<Tarea> cola) {
        this.cola = cola;
    }

    public void detener() {
        activo = false;
    }

    @Override
    public void run() {
        while (activo) {
            // size() de PriorityBlockingQueue es seguro para múltiples hilos
            System.out.println("[Monitor] Pendientes: " + cola.size());
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                break;
            }
        }
        System.out.println("[Monitor] Finalizado");
    }

    public static void main(String[] args) throws InterruptedException {
        PriorityBlockingQueue<Tarea> cola = new PriorityBlockingQueue<>();
        cola.add(new Tarea("Informe", 2));
        cola.add(new Tarea("Contrato", 1));

        MonitorPedidos monitor = new MonitorPedidos(cola);
        Thread hiloMonitor = new Thread(monitor);
        hiloMonitor.start();

        Thread.sleep(2500);
        monitor.detener(); // se detiene de forma segura desde el hilo principal
    }
}
```

Salida por consola:

```
[Monitor] Pendientes: 2
[Monitor] Pendientes: 2
[Monitor] Pendientes: 2
[Monitor] Finalizado
```

¿Qué se aplicó? Igual que `MonitorEstado` de la guía original: un hilo observador independiente que consulta el estado del sistema sin interferir en él, usando `volatile` para poder detenerse de forma segura desde otro hilo.

En resumen: a lo largo de estos 10 ejercicios se reforzaron los mecanismos clave de la semana 5: exclusión mutua con `synchronized` y `ReentrantLock`, coordinación de hilos con `wait()/notify()`, límite de acceso con `Semaphore`, orden de atención con `PriorityBlockingQueue`, visibilidad entre hilos con `volatile` y operaciones atómicas sin bloqueo con `AtomicInteger`. Estos son los mismos bloques usados en el simulador de restaurante (`CocinaSincronizada`, `CamareroConcurrencia` y `MonitorEstado`) de la guía de la experiencia.